



ERP Cashier

Système de Gestion de Caisse et de Point de Vente

Rapport de Projet Complet

Auteur :

Développeur Full Stack

Date :

5 février 2026



Table des matières

Résumé Exécutif

Le projet **ERP Cashier** est une solution complète de gestion de point de vente et de trésorerie (ERP), conçue pour moderniser et sécuriser les opérations de caisse au sein d'organisations multi-agences.

Objectifs Principaux

Ce système vise à résoudre les problèmes critiques liés à la gestion physique de la monnaie :

- **Sécurisation des flux financiers** : Traçabilité complète des ouvertures, fermetures et mouvements de fond.
- **Contrôle centralisé** : Supervision en temps réel des caissiers et des agences par les administrateurs.
- **Réconciliation rigoureuse** : Comparaison systématique entre les fonds théoriques et les comptages physiques (double comptage aveugle).

Technologies Clés

L'architecture technique repose sur une stack moderne garantissant performance et maintenabilité :

- **Frontend Backend** : Framework **Next.js 16** (App Router) pour une application web réactive et unifiée.
- **Base de Données** : **SQLite** via l'ORM **Prisma**, assurant une gestion robuste des relations complexes (Sessions, Mouvements, Profils).
- **Interface Utilisateur** : Styled avec **Tailwind CSS** pour une esthétique professionnelle et responsive.

Résultats Clés

Le projet livre une plateforme fonctionnelle permettant :

1. La gestion complète du cycle de vie des sessions de caisse (Ouverture → Opérations → Verrouillage → Fermeture).
2. L'administration des rôles et permissions (Superadmin, Admin d'Agence, Caissier).
3. La génération de rapports automatisés et de preuves d'audit (PDF).

Chapitre 1

Introduction

1.1 Contexte et Problématique

Dans le secteur de la vente au détail et des services financiers distribués, la gestion des espèces ("Cash Management") représente un défi opérationnel majeur. Les entreprises opérant sur plusieurs sites (agences) font face à des risques constants : erreurs de comptage, pertes inexplicables, et complexité de consolidation des données financières en fin de journée.

Les méthodes traditionnelles ou manuelles ne suffisent plus pour garantir l'intégrité des fonds. Le projet **ERP Cashier** naît de ce besoin impératif de digitaliser et de verrouiller les processus de caisse.

1.2 Objectifs du Projet

L'objectif principal est de fournir un outil unifié qui sert deux acteurs principaux :

- **Les Caissiers** : Une interface fluide pour enregistrer les opérations, déclarer les fonds et gérer leur session de travail sans ambiguïté.
- **Les Administrateurs** : Un tableau de bord de supervision ("Admin Panel") permettant de configurer les agences, d'assigner les caisses, et d'auditer les écarts de caisse en temps réel.

1.3 Périmètre du Travail

Le périmètre couvre l'intégralité de la chaîne de valeur logicielle :

1.3.1 Backend (API Data)

Le cœur du système est une API RESTful robuste exposée par Next.js. Elle gère :

- **L'Authentification Rôles** : Sécurisation des accès via des profils distincts (Person, CashierProfile, AdminProfile).
- **Logique Métier Complexe** : Gestion des états de session (ouverte/fermée/verrouillée) et calcul des soldes théoriques.
- **Persistance** : Modélisation précise des données (Prisma Schema) incluant les entités *CashRegister*, *Session*, *Movement*, et *Reconciliation*.

1.3.2 Frontend (Interfaces Utilisateur)

Deux interfaces distinctes ont été développées :

- **Back-office Admin** : Pour la gestion des référentiels (Agences, Utilisateurs, Caisses) et l'analyse des sessions.
- **Front-office Caissier** : Optimisé pour la saisie rapide et la fiabilité des opérations quotidiennes.

Chapitre 2

Architecture Technique

2.1 Vue d'ensemble de la Stack

L'application **ERP Cashier** repose sur une architecture monolithique moderne, tirant parti de la puissance de l'écosystème JavaScript/TypeScript unifié.

2.1.1 Frontend & Backend (Next.js)

Nous utilisons **Next.js 16** avec l'architecture *App Router*. Ce choix stratégique permet :

- **Server Actions** : Traitement sécurisé des soumissions de formulaires et des mutations de données directement côté serveur, sans exposer d'endpoints API publics inutiles.
- **Server Components (RSC)** : Rendu initial rapide et optimisation SEO, tout en réduisant la taille du bundle JavaScript envoyé au client.
- **API Routes** : Une couche API intégrée pour les besoins spécifiques (ex : endpoints pour des intégrations tierces ou des appels AJAX complexes).

2.1.2 Base de Données (SQLite & Prisma)

Pour la persistance des données, nous avons opté pour **SQLite** couplé à l'ORM **Prisma**.

- **SQLite** : Offre une simplicité de déploiement (fichier unique) et des performances excellentes pour les volumes de données visés (mono-instance).
- **Prisma** : Garantit la sécurité des types (Type-Safety) de bout en bout, réduisant drastiquement les erreurs d'exécution liées aux requêtes SQL.

2.2 Modélisation des Données

Le schéma de base de données est conçu pour garantir l'intégrité financière. Voici les entités critiques :

2.2.1 Gestion des Utilisateurs

1. **Person** : L'entité racine représentant un individu physique.
2. **CashierProfile** : Extension pour les employés de caisse, contenant leurs identifiants de connexion spécifiques (PIN ou mot de passe).
3. **AdminProfile** : Extension pour les gestionnaires, conférant des droits d'accès au Back-office.

2.2.2 Gestion des Fonds (Session Mouvements)

Le cœur du système repose sur le concept de **Session** :

- **Session** : Représente une période de travail d'un caissier sur une caisse donnée. Elle capture le fond de caisse initial (*openingBalance*) et final.
- **Movement** : Chaque transaction (entrée/sortie d'argent) est un enregistrement immuable lié à une session.
- **Reconciliation** : Entité de vérification où l'administrateur ou le système valide la correspondance entre le solde théorique (calculé via les mouvements) et le solde physique (compté).

2.3 Sécurité

La sécurité est assurée à plusieurs niveaux :

- **Authentification** : Sessions sécurisées via cookies HTTP-only.
- **Autorisation** : Middleware Next.js vérifiant les rôles avant chaque accès aux routes protégées (/admin/* vs /pos/*).
- **Traçabilité** : Chaque action critique est logguée avec l'ID de l'utilisateur et un timestamp.

Chapitre 3

Implémentation et Fonctionnalités

3.1 Workflow Caissier

Le module "Point de Vente" (POS) est conçu pour guider le caissier à travers un processus rigoureux, minimisant les risques d'erreurs humaines.

3.1.1 Ouverture de Session

Avant de pouvoir enregistrer la moindre transaction, le caissier doit impérativement ouvrir une session.

1. **Identification** : Le caissier se connecte avec ses identifiants personnels.
2. **Comptage Initial** : Il doit saisir le montant exact présent en caisse (le fond de caisse).
3. **Validation** : Le système crée une nouvelle *Session* en base de données avec le statut OPEN.

3.1.2 Opérations Quotidiennes

Une fois la session ouverte, le caissier accède à l'interface de vente. Les fonctionnalités clés incluent :

- **Encaissement** : Enregistrement rapide des ventes avec calcul automatique du rendu de monnaie.
- **Mouvements de Fond** : Possibilité d'ajouter (apport) ou de retirer (prélèvement) de l'argent pour des raisons opérationnelles, avec justification obligatoire.
- **Visualisation** : Un récapitulatif en temps réel des transactions de la session courante.

3.1.3 Clôture et Réconciliation

La fin de journée est l'étape la plus critique :

1. **Verrouillage** : Le caissier initie la fermeture. La caisse passe en mode "Verrouillé", bloquant toute nouvelle transaction.
2. **Comptage Aveugle** : Le système demande au caissier de saisir le montant final compté, SANS lui afficher le montant théorique attendu. C'est le principe du "Blind Count".
3. **Fermeture** : La session est clôturée. Si un écart existe entre le théorique et le réel, il est enregistré et flagué pour l'administrateur.

3.2 Supervision Administrateur

Le panneau d'administration offre une vue d'ensemble ("God Mode") sur l'activité des agences.

3.2.1 Tableau de Bord en Temps Réel

Les administrateurs peuvent voir l'état de chaque caisse (Ouverte/Fermée) et le caissier actuellement connecté. Des alertes visuelles signalent les sessions ouvertes depuis trop longtemps ou présentant des anomalies.

3.2.2 Audit et Rapports

L'historique complet est accessible :

- **Détail des Sessions** : Accès à la liste exhaustive des mouvements d'une session passée.
- **Traçabilité des Écarts** : Identification rapide des caissiers ou des périodes générant des écarts de caisse récurrents.

Chapitre 4

Conclusion et Perspectives

Le projet **ERP Cashier** a atteint ses objectifs initiaux en livrant une solution de gestion de caisse sécurisée, moderne et centralisée. En remplaçant les processus manuels par un système numérique rigoureux, l'application réduit considérablement les risques d'erreurs et de fraudes.

L'architecture choisie (Next.js + SQLite) s'est avérée être un excellent compromis entre rapidité de développement, performance et maintenabilité. La séparation claire entre le Front-office (Caissier) et le Back-office (Admin) garantit une expérience utilisateur adaptée à chaque profil.

Perspectives d'Évolution

Bien que le système soit pleinement opérationnel, plusieurs pistes d'amélioration sont envisagées pour les versions futures :

- **Intégration TPE (Terminaux de Paiement)** : Connecter directement l'application aux terminaux bancaires pour automatiser la saisie des montants payés par carte, éliminant une autre source d'erreur de saisie.
- **Application Mobile** : Développer une version mobile (PWA ou React Native) pour permettre aux administrateurs de surveiller les agences depuis leur smartphone en déplacement.
- **Analyse Prédictive** : Utiliser l'historique des données pour prédire les besoins en fond de caisse par jour et par agence, optimisant ainsi la trésorerie immobilisée.

Ce projet constitue donc une fondation solide pour la transformation digitale de la gestion financière du réseau d'agences.